© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2025
S. K. Sahu, *Generative AI-Driven Application Development with Java*
https://doi.org/10.1007/979-8-8688-1609-3_5

5. Power Tools: LangChain4j Quick-Start

Satej Kumar Sahu[1]
(1) Berhampur, Odisha, India

This chapter introduces LangChain4j, the Java framework designed to simplify the integration of LLMs into Java applications. It covers the installation, setup, and use of key features and capabilities.

Introduction

In this chapter, you will start our journey by beginning to work hands-on with LLM frameworks, particularly LangChain4j. While doing so, you will keep learning about many of the concepts and processes in building LLM applications. Let's start with a simple application that just invokes an LLM by passing it some input and receiving some output. Then you improve the output and architecture of the application by using prompt engineering and then retrieval-augmented generation (RAG). The application you will be building will help users get customer support, book tickets, and various other actions that would be applicable in an airline management system, but through a chat interface. You will not be concerned with the user interface and will be limiting yourself to the application programming interface (API) interfaces.

What Is LangChain4j?

LangChain is a popular LLM orchestration framework. It contains modularized components that can be connected through different steps, from processing your inputs to fetching data from vector databases to invoking tools. You will learn about all these aspects as you follow along in the chapter. LangChain is written in Python, and LangChain4j means LangChain for Java. LangChain4j is the Java language counterpart for LangChain. I will be using the term LangChain4j going ahead. Where explicitly needed, I will refer to LangChain when particular differences or features need to be pointed out.

LangChain4j provides a unified framework and API interface to help you build chatbots, assistants, and other aspects around LLMs. It integrates smoothly into frameworks like Spring Boot and Quarkus. It provides a wide range of features applicable to LLM frameworks like prompt templating, chat memory management, output parsing, and other such functionalities. You will learn about all of these later in the chapter. You can find more information at **https://docs.langchain4j.dev/**.

Installation and Setup of LangChain4j

These are the prerequisites:

- Java 17+
- Gradle/Maven
- IntelliJ/VS Code/any relevant integration development environment (IDE)

Start the setup by initializing a basic simple Spring Boot application using the Spring Initializer at https://start.spring.io/, as shown in Figure 5-1. For the following instructions, I will use Gradle. The steps could be easily replaced with Maven-based instructions.

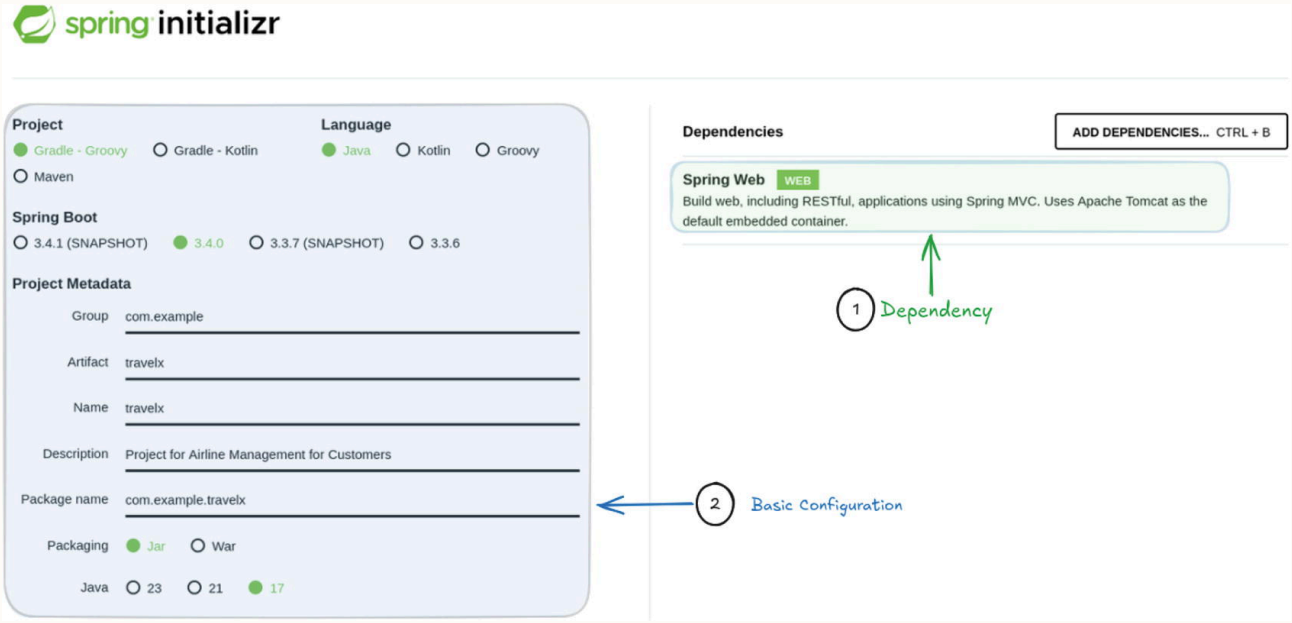


Figure 5-1 Spring Initializr dependencies

We will be working for a hypothetical company named TravelX. In the very first iteration, you will build an API endpoint that can be used by front-end teams. This API takes input in the form of text and gives back a response from LLM. In this iteration, you want to expose an interface to customers where they can chat with your bot.

As the first step, you will unzip the sample Spring Boot zip downloaded.

```
unzip travelx.zip
cd travelx
./gradlew build
```

Let's now add the LangChain4j dependency to the project. If you are using Maven, please add the following dependency to the pom.xml file:

```
<dependency>
    <groupId>dev.langchain4j</groupId>
    <artifactId>langchain4j-open-ai</artifactId>
    <version>0.36.2</version>
</dependency>
<dependency>
    <groupId>dev.langchain4j</groupId>
    <artifactId>langchain4j</artifactId>
    <version>0.36.2</version>
</dependency>
```

For Gradle, please add the following to the build.gradle file:

```
implementation 'dev.langchain4j:langchain4j-open-ai:0.36.2'
implementation 'dev.langchain4j:langchain4j:0.36.2'
```

File: build.gradle

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '3.4.0'
    id 'io.spring.dependency-management' version '1.1.6'
}
group = 'com.example'
version = '0.0.1-SNAPSHOT'
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(17)
    }
}
repositories {
    mavenCentral()
}
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'dev.langchain4j:langchain4j-open-ai:0.36.2'
```

```
    implementation 'dev.langchain4j:langchain4j:0.36.2'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
tasks.named('test') {
    useJUnitPlatform()
}
```

Initial Project Setup: Creating Your First LangChain4j Application

You will integrate with OpenAI API, which is an API interface for LLM models created by OpenAI. Figure 5-2 shows the project architecture.

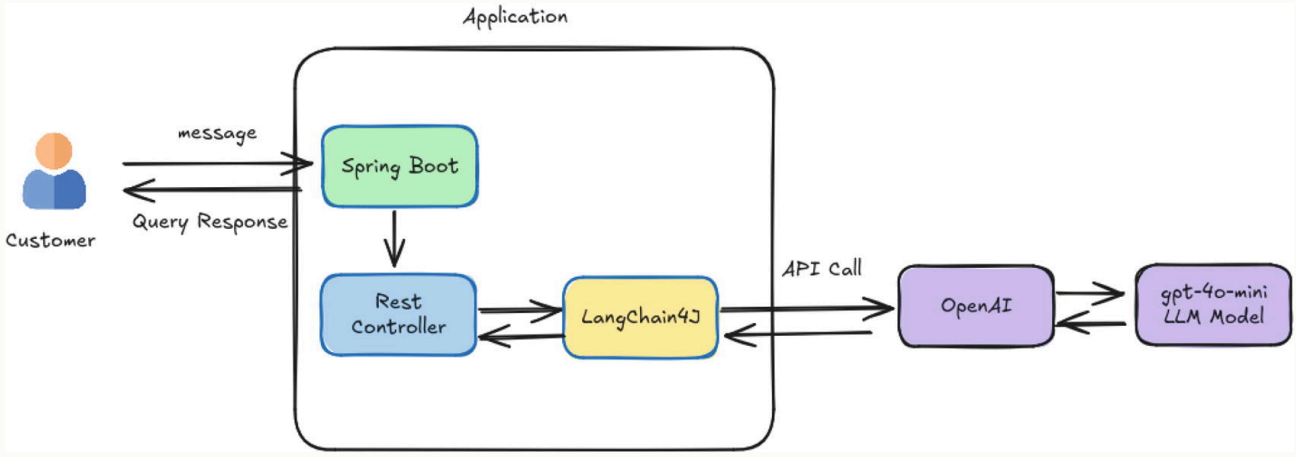


Figure 5-2 Initial project architecture

For this, you will need an OpenAI API key. LangChain4j provides an OpenAI key called *demo* for testing purposes that you can use. Let's set the key in application.properties, although the best practice would be to set it in your environment variables.

File: application.properties

```
openai.api-key=demo
```

Next, you will create a bean configuration for connecting to OpenAI and choosing a language model. You can choose gpt-4o-mini, which is a small language model and gives faster response. Let's create a file called LangchainConfig.java with the following code. It imports the OpenAiChatModel from LangChain4j and builds a connection config to OpenAI by passing in the API key and the language model you want to invoke for the requests to OpenAI.

```
package com.example.travelx;
```

```
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import dev.langchain4j.model.openai.OpenAiChatModel;
@Configuration
public class LangchainConfig {
   @Value("${openai.api-key}")
   private String apiKey;
   @Bean
   public OpenAiChatModel openAiChatModel() {
      return OpenAiChatModel.builder()
            .apiKey(apiKey)
            .modelName("gpt-4o-mini")
            .build();
   }
}
```

To handle chat requests from the UI, let's create a simple POJO to define the structure of the request in a file called ChatRequest.java.

```
package com.example.travelx;
class ChatRequest {
   private String message;
   public String getMessage() {
      return message;
   }
   public void setMessage(String message) {
      this.message = message;
   }
}
```

Once you have the previous two files set up, the final goal is to create a REST controller that leverages these two files. Name the file ChatController.java and add the following content in it:

```
package com.example.travelx;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import dev.langchain4j.model.openai.OpenAiChatModel;
@RestController
@RequestMapping("/api/chat")
public class ChatController {
  private final OpenAiChatModel chatModel;
  @Autowired
```

```
    public ChatController(OpenAiChatModel chatModel) {
        this.chatModel = chatModel;
    }
    @PostMapping
    public String chat (@RequestBody ChatRequest chatRequest) {
        return chatModel.generate(chatRequest.getMessage());
    }
}
```

Let's restart the Spring Boot Application and issue a POST request against it at /api/chat. You should receive a response from the API saying "Hello, World!" Hurray, you have your first interaction with the world of LLMs and your first requirement.

```
curl -X POST http://localhost:8080/api/chat -H "Content-Type: application/json" -d '{
 "message": "Say Hello World"
}'
Hello, World!
```

Conclusion

In this chapter, you explored the foundational aspects of integrating LangChain4j into a Java application, focusing on its use within the context of a Spring Boot project. LangChain4j, the Java counterpart to LangChain, provides a seamless way to integrate LLMs into Java-based applications with features such as prompt engineering, chat memory, and output parsing.

We began by understanding LangChain4j's purpose and capabilities, emphasizing its utility in developing modular and scalable applications. The chapter provided step-by-step instructions for setting up a basic Spring Boot project and integrating LangChain4j dependencies. You configured the OpenAI API connection, chose a lightweight language model (gpt-4o-mini), and built a simple chatbot endpoint.

Through a practical example, you implemented a REST controller that processes user inputs, invokes the LLM, and returns generated responses. By running the application and issuing a test request, you successfully saw a chatbot's interaction, completing your first integration with LangChain4j.

This chapter has equipped you with the knowledge to get started with LangChain4j, laying a strong foundation for developing advanced applications involving LLMs. The next steps involve exploring more advanced features like RAG, enhanced prompt engineering, and integrating tools for richer interactions.

Previous chapter

< 4. Bring Your Own Model: Self-Hosting with Ollama

Next chapter

6. Integrating LLMs with Java Applications >